

app.add_media_files 全面详细解析

`app.add_media_files()` 是 NiceGUI 框架中针对媒体文件流式传输设计的核心方法，隶属于 `nicegui.app` 模块。其核心作用是将服务器本地媒体文件目录映射为支持**字节范围请求 (Byte-Range Requests)** 的 Web 端点，实现媒体文件的增量加载、断点续传和播放进度跳转，解决普通静态文件暴露方式（如 `add_static_files()`）无法满足音视频等媒体文件流式播放的问题。

一、核心功能与适用场景

1. 核心价值

- 流式传输支持**：基于 HTTP 字节范围请求 (Byte-Range) 实现媒体文件的分段加载，浏览器可按需请求文件的部分内容（如视频播放时跳转到指定时间点），而非一次性下载整个文件；
- 媒体适配性**：针对音视频、大体积图片等媒体文件优化，兼容浏览器原生的媒体播放逻辑（如 `<video>` / `<audio>` 标签的进度控制）；
- 简化媒体资源管理**：无需手动编写流式传输的 FastAPI 路由 / 响应逻辑，框架自动处理 MIME 类型识别、范围请求解析、响应头配置等；
- 安全边界**：明确限定仅暴露非敏感媒体文件（映射后的文件对所有访问者开放）。

2. 典型适用场景

- 前端播放本地视频文件（MP4、WebM、MKV 等），支持进度条跳转、倍速播放；
- 播放本地音频文件（MP3、WAV 等），支持暂停 / 继续、进度调整；
- 加载大体积图片（如高清海报、全景图），实现渐进式加载；
- 替代 `add_static_files()` 处理媒体文件，解决“视频无法拖动进度条”“音频只能从头播放”等问题。

3. 与 `add_static_files()` 的核心差异

特性	<code>app.add_media_files()</code>	<code>app.add_static_files()</code>
传输方式	流式传输（支持字节范围请求）	一次性传输（不支持字节范围请求）
适配文件类型	音视频、大体积媒体文件	普通静态文件（代码、CSS、小图片等）
播放体验	支持进度跳转、断点续传、增量加载	媒体文件需完整下载后才能播放，无进度控制
响应头优化	自动添加 <code>Accept-Ranges / Content-Range</code> 等媒体专用头	仅返回基础 <code>Content-Type / Cache-Control</code>
性能（大文件）	低内存占用（分段加载）	高内存占用（一次性加载整个文件）

4. 关联方法对比

`app.add_media_files()` 是批量暴露媒体目录的方式，NiceGUI 提供单文件专用方法适配不同场景：

方法名	功能	适用场景
<code>add_media_files()</code>	暴露整个本地目录，支持媒体流式传输	批量提供音视频、大体积图片等媒体文件
<code>add_media_file()</code>	暴露单个本地文件，支持媒体流式传输	仅需共享 1-2 个媒体文件（如单个核心视频）
<code>add_static_files()</code>	暴露整个目录，普通静态传输	非媒体类小文件（代码、配置、小图片）

二、参数详解

`app.add_media_files()` 接收 2 个必填参数，无可选参数（核心逻辑聚焦媒体流式传输，简化配置）：

参数名	类型	是否必填	核心说明
<code>url_path</code>	字符串	是	前端访问媒体文件的 URL 路径， 必须以斜杠 / 开头 （如 <code>/my_videos</code> 、 <code>/media/audio</code> ）；最终文件访问路径为 <code>url_path + 本地文件相对路径</code> （例： <code>/my_videos/clouds.mp4</code> 对应本地 <code>media/clouds.mp4</code> ）
<code>local_directory</code>	字符串 / Path	是	服务器本地媒体文件目录的路径（支持字符串路径或 <code>pathlib.Path</code> 对象）；相对路径基于运行 <code>main.py</code> 的目录，绝对路径需确保脚本有读取权限

参数使用示例（补充）

```
from nicegui import app
from pathlib import Path

# 用 Path 对象指定本地媒体目录，映射到 /media/audio 路径
audio_dir = Path('media/audio')
app.add_media_files(url_path='/media/audio', local_directory=audio_dir)

# 用字符串指定目录，映射到 /media/imgs 路径（大体积图片）
app.add_media_files(url_path='/media/imgs', local_directory='media/hd_images')
```

三、使用示例与访问逻辑

1. 基础使用示例（官方示例拆解）

```
import httpx
from nicegui import app, ui
from pathlib import Path

# 1. 准备本地媒体目录和文件
media = Path('media')
media.mkdir(exist_ok=True) # 确保目录存在
# 下载示例视频到本地 media 目录
r = httpx.get('https://cdn.coverr.co/videos/coverr-cloudy-sky-2765/1080p.mp4')
(media / 'clouds.mp4').write_bytes(r.content)

# 2. 核心配置：将本地 media 目录映射为 /my_videos 端点，支持媒体流式传输
app.add_media_files('/my_videos', media)

# 3. 前端播放视频（直接引用映射后的 URL，支持进度跳转）
ui.video('/my_videos/clouds.mp4')

ui.run()
```

2. 访问逻辑说明

启动应用后（默认端口 8080）：

- 前端 `<video>` 标签请求 `http://localhost:8080/my_videos/clouds.mp4`；
- NiceGUI 检测到该请求来自 `add_media_files()` 映射的端点，自动启用流式传输逻辑：
 1. 解析浏览器发送的 `Range` 请求头（如 `Range: bytes=0-1023`）；
 2. 读取本地文件的对应字节范围内容；
 3. 返回包含 `Content-Range / Accept-Ranges: bytes` 的响应头；
- 浏览器可根据播放进度多次请求不同字节范围，实现进度条拖动、断点续传。

3. 进阶示例：播放音频文件

```
from nicegui import app, ui
from pathlib import Path

# 准备音频目录和文件（假设本地有 media/audio/music.mp3）
audio_dir = Path('media/audio')
audio_dir.mkdir(parents=True, exist_ok=True)

# 映射音频目录到 /media/audio 端点
app.add_media_files('/media/audio', audio_dir)

# 前端播放音频，支持进度控制
ui.audio('/media/audio/music.mp3').props('controls')
```

四、关键注意事项

1. 安全约束（核心）

- 仅存放**非安全关键文件**：映射后的媒体文件对所有访问者开放，禁止暴露包含隐私 / 敏感信息的媒体文件（如用户私密视频、带敏感数据的图片）；
- 权限控制：确保运行 Python 脚本的用户对 `local_directory` 有读取权限（否则媒体文件无法加载）；
- 目录范围限制：仅暴露指定目录内的文件，禁止通过 URL 路径遍历（如 `../`）访问目录外的文件（框架已做路径安全校验）。

2. 路径规则

- `url_path` 必须以 `/` 开头：如 `/my_videos` 合法，`my_videos` 或 `./my_videos` 非法；
- 避免 URL 路径冲突：若已用 `/media` 作为媒体目录端点，不可再将 `/media` 作为页面路由（如 `ui.route('/media', ...)`），否则媒体请求会被路由拦截；
- 支持 `Path` 对象：推荐使用 `pathlib.Path` 处理目录路径，提升跨平台兼容性（Windows/macOS/Linux 路径格式统一）。

3. 媒体文件兼容性

- 浏览器对媒体格式的支持：`ui.video/ui.audio` 依赖浏览器原生解码能力，建议使用通用格式（视频：MP4；音频：MP3），避免 MKV、FLAC 等兼容性差的格式；
- 大文件处理：无文件大小限制（流式传输按分段加载），但需确保服务器磁盘有足够空间存储媒体文件。

4. 版本兼容性

- `app.add_media_files()` 是 NiceGUI 2.0.0+ 核心版本支持的功能，全版本无参数变更，兼容性稳定；
- 若使用低版本（<2.0.0），需手动编写 FastAPI 路由实现流式传输（框架无封装方法）。

5. 性能优化

- 媒体文件存储：建议将大体积媒体文件放在高速磁盘（如 SSD），提升分段加载速度；
- 缓存控制：媒体文件默认无特殊缓存配置，若需缓存可结合反向代理（如 Nginx）设置 `Cache-Control` 头，减少重复请求。

五、核心总结

`app.add_media_files()` 是 NiceGUI 专为媒体文件优化的静态资源暴露方法，核心优势是支持流式传输和字节范围请求，完美适配音视频播放的进度控制需求。使用时需重点关注：

1. 仅暴露非敏感媒体文件，确保数据安全；
2. 遵循 `url_path` 以 `/` 开头的规则，避免路径冲突；
3. 优先用于音视频、大体积图片，普通静态文件仍用 `add_static_files()`；
4. 推荐使用 `pathlib.Path` 处理本地目录路径，提升跨平台兼容性。

NiceGUI 中 `add_media_files` 方法的全维度解析

`add_media_files` 是 NiceGUI 专为**媒体文件（音视频、流媒体）** 优化的静态资源托管方法，是 `add_static_files` 的媒体专用版本，底层针对音视频的流式播放、分段加载、媒体 MIME 类型自动适配做了深度优化，适用于托管需要浏览器原生播放的音频（MP3/WAV）、视频（MP4/WEBM）、流媒体（HLS/DASH）等资源。

一、核心作用与适用场景

1. 核心作用

- 托管本地媒体文件夹到指定 URL 路径，支持浏览器原生音视频播放；
- 优化媒体文件的 HTTP 响应头（如 `Accept-Ranges`、`Content-Length`），支持断点续传、分段加载；
- 自动适配媒体文件的 MIME 类型（如 MP4→`video/mp4`、MP3→`audio/mpeg`）；
- 支持流媒体协议（HLS/DASH）的分片文件托管，兼容主流播放器。

2. 典型适用场景

场景	示例
网页播放本地视频	嵌入 <code>ui.video</code> 播放 <code>/media/videos/demo.mp4</code>
网页播放本地音频	嵌入 <code>ui.audio</code> 播放 <code>/media/audio/song.mp3</code>
托管流媒体分片文件	播放 HLS 格式的 <code>.m3u8</code> 和 <code>.ts</code> 分片文件
支持视频断点续传	大视频文件播放时暂停 / 续播、拖拽进度条
移动端媒体播放适配	兼容手机浏览器的原生音视频播放控件

二、基本语法与使用方式

1. 基础语法

```
from nicegui import ui, app

# 核心方法：映射本地媒体文件夹到URL路径
app.add_media_files(
    url_path: str, # 访问媒体文件的URL路径（如'/media'）
    local_directory: str, # 本地媒体文件夹路径（绝对/相对）
    # 可选高级参数（与add_static_files一致，新增媒体专属优化）
    cache_control: str | None = 'max-age=86400', # 默认缓存1天
    follow_symlink: bool = False,
    check_dir: bool = True,
)

# 最简示例：映射./media文件夹到/media URL
app.add_media_files('/media', './media')
```

ui.run()

2. 核心参数详解

参数名	类型	取值说明	默认值
url_path	str	访问媒体资源的 URL 前缀（必须以 / 开头，如 /media、/videos）	无（必填）
local_directory	str	本地媒体文件夹路径：- 相对路径：相对于应用启动目录- 绝对路径：直接指定（如 /home/user/media）	无（必填）
cache_control	str / None	媒体文件缓存策略（默认 max-age=86400 即 1 天，音视频建议长缓存）	max-age=86400
follow_symlink	bool	是否跟随符号链接（安全风险，默认关闭）	False
check_dir	bool	启动时检查文件夹是否存在（不存在则抛异常）	True

3. 核心差异（与 add_static_files 对比）

add_media_files 是 add_static_files 的媒体专用增强版，核心优化点：

特性	add_media_files	add_static_files
断点续传	原生支持（返回 Accept-Ranges: bytes）	需手动配置
媒体 MIME 类型	精准适配（如 MP4→video/mp4、WEBM→video/webm）	基础推断（可能不准确）
分段加载	优化响应头，支持音视频拖拽进度条	无优化，大视频拖拽会卡顿
流媒体协议	兼容 HLS/DASH 分片文件托管	无特殊兼容
默认缓存策略	1 天（适合静态媒体文件）	None（无缓存）

4. 基础示例：播放本地视频 / 音频

步骤 1：创建本地媒体文件夹结构

```
your_app/
├─ main.py          # 应用入口
├─ media/           # 媒体文件夹
│   ├─ videos/      # 视频子文件夹
│   │   └─ demo.mp4 # 视频文件
│   └─ audio/        # 音频子文件夹
│       └─ song.mp3  # 音频文件
```

步骤 2：编写代码托管并播放媒体

```
from nicegui import ui, app
import os

# 确保媒体文件夹存在
os.makedirs('./media/videos', exist_ok=True)
os.makedirs('./media/audio', exist_ok=True)

# 映射媒体文件夹到/media URL
app.add_media_files('/media', './media')

# 播放视频（支持断点续传、拖拽进度条）
ui.label('本地视频播放').classes('text-x1 my-2')
ui.video('/media/videos/demo.mp4').classes('w-full max-w-2x1 mx-auto')

# 播放音频
ui.label('本地音频播放').classes('text-x1 my-4')
ui.audio('/media/audio/song.mp3').classes('w-full max-w-2x1 mx-auto')

ui.run()
```

5. 进阶示例：托管 HLS 流媒体文件

HLS (HTTP Live Streaming) 是主流流媒体协议，由 `.m3u8` 索引文件和 `.ts` 分片文件组成，`add_media_files` 可完美托管：

```
from nicegui import ui, app
import os

# 流媒体文件夹（包含.m3u8和.ts文件）
HLS_DIR = './media/hls'
os.makedirs(HLS_DIR, exist_ok=True)

# 映射流媒体文件夹（缓存策略设为no-cache，因分片文件会更新）
app.add_media_files('/hls', HLS_DIR, cache_control='no-cache')

# 播放HLS流媒体（需浏览器支持HLS，如Chrome/Firefox）
ui.label('HLS流媒体播放').classes('text-x1 my-2')
# 注意：HLS需指定type为application/x-mpegURL
ui.video('/hls/stream.m3u8', type='application/x-mpegURL').classes('w-full max-w-2x1 mx-auto')

ui.run()
```

三、关键特性与注意事项

1. 断点续传与分段加载

`add_media_files` 自动返回以下 HTTP 响应头，支持音视频断点续传：

- `Accept-Ranges: bytes`：告知浏览器支持字节范围请求；
- `Content-Length`：返回文件总大小；
- `Content-Range`：响应分段请求时返回当前分段范围。

这使得用户播放大视频时，可随意拖拽进度条，浏览器只会请求对应时间段的字节数据，而非整个文件。

2. 媒体 MIME 类型精准适配

`add_media_files` 内置了完整的媒体文件 MIME 类型映射，常见类型如下：

文件扩展名	MIME 类型
.mp4	video/mp4
.webm	video/webm
.mkv	video/x-matroska
.mp3	audio/mpeg
.wav	audio/wav
.ogg	audio/ogg
.m3u8	application/x-mpegURL
.ts	video/mp2t
.flac	audio/flac

若遇到不识别的媒体类型，可手动通过 `add_static_files` 并指定 `mime_type` 补充，或升级 NiceGUI 版本。

3. 路径处理与跨平台兼容

- 与 `add_static_files` 一致，推荐使用绝对路径避免工作目录问题：

```
import os
from nicegui import ui, app

# 获取脚本所在目录
SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
# 拼接媒体文件夹绝对路径
MEDIA_DIR = os.path.join(SCRIPT_DIR, 'media')
app.add_media_files('/media', MEDIA_DIR)
```

- Windows 路径需使用原始字符串 (`r'C:\media'`) 或 `os.path.join`，避免转义符问题。

4. 大媒体文件优化

- 对于 GB 级别的大视频文件，`add_media_files` 的流式传输特性可避免加载整个文件到内存；
- 建议将大媒体文件存储在高速磁盘（如 SSD），或结合 CDN 使用；
- 生产环境可开启 Gzip/Brotli 压缩（但音视频本身已压缩，效果有限，建议关闭）。

5. 安全注意事项

- 禁止映射系统目录或包含敏感信息的文件夹，避免泄露；
- `follow_symlink=False`：默认不允许访问符号链接，防止跨目录访问；
- 若需限制媒体文件访问（如仅登录用户可播放），需结合 NiceGUI 的认证机制：

```
from nicegui import ui, app
from starlette.middleware.authentication import AuthenticationMiddleware

# 简易认证中间件（示例）
async def auth_check(request, call_next):
    if request.url.path.startswith('/media/') and not request.cookies.get('token'):
        return ui.Response(status_code=401, content='未授权')
    return await call_next(request)

app.add_middleware(AuthenticationMiddleware, backend=auth_check)
app.add_media_files('/media', './media')
```

6. 常见陷阱

- **视频播放卡顿 / 无法拖拽**：若使用 `add_static_files` 托管视频会出现此问题，需替换为 `add_media_files`；
- **HLS 流媒体 404**：确保 `.m3u8` 文件中的 `.ts` 分片路径与 URL 路径一致，避免相对路径错误；
- **移动端播放失败**：检查媒体编码格式（移动端仅支持 MP4/H.264、MP3 等通用格式）；
- **缓存导致媒体更新不生效**：修改媒体文件后，若浏览器仍播放旧版本，需调整 `cache_control` 为 `no-cache`，或给 URL 加版本参数（如 `/media/videos/demo.mp4?v=2`）。

7. 与 `ui.download` 的配合

- `add_media_files`：用于**在线播放**媒体文件（浏览器原生控件播放）；
- `ui.download`：用于**下载**媒体文件（触发浏览器下载弹窗）；
- 实战组合示例：

```
# 播放+下载按钮组合
ui.video('/media/videos/demo.mp4').classes('w-full max-w-2xl mx-auto')
ui.button('下载该视频', on_click=lambda:
ui.download('/media/videos/demo.mp4')).classes('mt-2')
```

四、实战场景示例（完整媒体托管与播放）

```
from nicegui import ui, app
import os

# 1. 定义绝对路径（避免工作目录问题）
SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
MEDIA_DIR = os.path.join(SCRIPT_DIR, 'media')
VIDEO_DIR = os.path.join(MEDIA_DIR, 'videos')
AUDIO_DIR = os.path.join(MEDIA_DIR, 'audio')
HLS_DIR = os.path.join(MEDIA_DIR, 'hls')

# 2. 创建媒体文件夹
for dir_path in [MEDIA_DIR, VIDEO_DIR, AUDIO_DIR, HLS_DIR]:
    os.makedirs(dir_path, exist_ok=True)

# 3. 托管媒体文件（不同缓存策略）
# 视频/音频：缓存1天（静态文件）
app.add_media_files('/media', MEDIA_DIR)
# 流媒体：禁用缓存（分片文件会更新）
app.add_media_files('/hls', HLS_DIR, cache_control='no-cache')

# 4. UI布局：分类播放媒体
with ui.tabs() as tabs:
    tab_video = ui.tab('本地视频')
    tab_audio = ui.tab('本地音频')
    tab_hls = ui.tab('HLS流媒体')

with ui.tab_panels(tabs, value=tab_video).classes('w-full max-w-3xl mx-auto'):
    # 本地视频面板
    with ui.tab_panel(tab_video):
        ui.label('MP4视频播放（支持断点续传）').classes('text-lg mb-2')
        ui.video('/media/videos/demo.mp4').classes('w-full')
        ui.button('下载视频', on_click=lambda:
ui.download('/media/videos/demo.mp4')).classes('mt-2')

    # 本地音频面板
    with ui.tab_panel(tab_audio):
        ui.label('MP3音频播放').classes('text-lg mb-2')
        ui.audio('/media/audio/song.mp3').classes('w-full')
        ui.button('下载音频', on_click=lambda:
ui.download('/media/audio/song.mp3')).classes('mt-2')

    # HLS流媒体面板
    with ui.tab_panel(tab_hls):
        ui.label('HLS流媒体播放（.m3u8）').classes('text-lg mb-2')
        ui.video('/hls/stream.m3u8', type='application/x-mpegURL').classes('w-full')

ui.run(port=8080, title='媒体文件托管与播放示例')
```

